Problem Summary

Given an array of integers 'nums' and an integer 'target' return the indices of the two numbers that add up to 'target'.

You may assume exactly one solution exists, and you cannot use the same index twice

Key Idea

Store each number and its index in a hash map.

Then for each number:

→ Compute the value needed to reach target

→ check if that value exists in the map

→ make sure it is not the same index

Diagram Examples

nums = [2, 7, 11, 15], target = 9

hashmap = {2:0, 7:1, 11:2, 15:3}

nums = [2, 3, 3, 2]

hashmap = {2:3, 3:2}

latest occurence in array

Approach

1) Create empty hash map

2) First pass : Store each number with its index

3) Second pass: Compute needed value = target - nums[i]

4) If needed exists in map and is not the same index : return [i, map[needed]]

Time Complexity : O(n) → first loop to store in hashmap is O(n), second loop to find indices O(n)

Space Complexity : O(n) → for hash map